

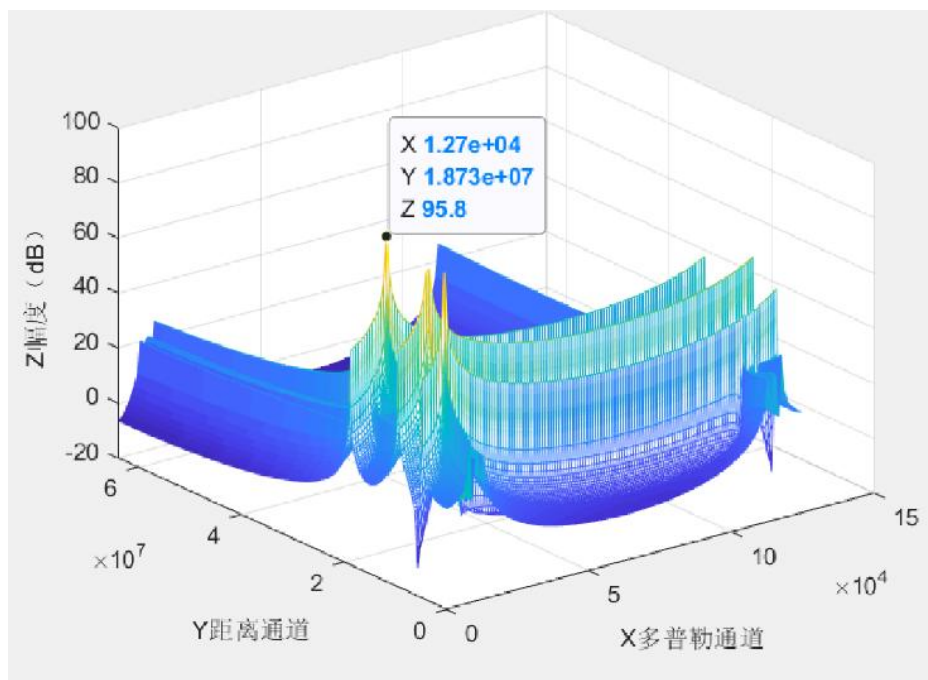
2026 Spring SME212 Lab Project

特别注意

- 个人独立完成，严禁小组合作或代写。
- 提交代码将进行查重比对，请确保代码的原创性。
- FPGA Demo Due: 2026/6/18 17:00 (第 17 周周四下午)
- Project Report Due: 2026/6/21 23:59 (第 17 周周日晚)

I. 项目背景

在数字信号处理中，二维快速傅里叶变换(2D-FFT)广泛应用于图像处理、雷达信号分析和通信系统中。FFT 结果中的峰值位置通常对应于信号的主要频率分量，对于后续的信号分析和处理至关重要。传统的软件实现方式在实时处理的场景中存在性能瓶颈，而基于 FPGA 的硬件实现可以通过系统架构、定制电路实现更低的延迟。



针对上述现实问题，本项目将使用 Verilog HDL 设计一个简化版的峰值检测器，能够实时检测 64x64 二维数据（如 2D-FFT 的幅度）中的显著峰值，并将其部署于 FPGA 开发板上进行 Demo 演示。

II. 项目细节

2.1) 项目包括 4 个任务：

- 背景调研：利用 UDA（Univista Design Assistant，数字设计 AI 智能平台）课程工具进行深入的项目背景调研，比较 2 到 3 种不同的实现方式（例如 CPU、FPGA、ASIC 等），特别关注 FPGA 方案在吞吐量、延时等方面的独特优势。
- 系统设计：在完成初步调研后，基于给定代码框架（详见 Blackboard），整理出模块化设计方案和检测流程图，并阐述个人的设计思路。
- 子模块实现与验证：根据前面的整体设计方案，编写和完善各个子模块的 Verilog 代码，并对关键子模块（特别是 Core 模块）进行仿真验证。其中，Core 模块可使用 UDA 课程工具进行辅助设计开发。
- 系统集成与演示：在 FPGA 上实现峰值检测功能，并现场 Demo 演示。

2.2) Core 模块说明

```

1  module Core #(
2      parameter ROWS = 64,
3      parameter COLS = 64,
4      parameter ROWS_ADDR_WIDTH = 6, // log2(ROWS)
5      parameter COLS_ADDR_WIDTH = 6, // log2(COLS)
6      parameter BRAM_ADDR_WIDTH = ROWS_ADDR_WIDTH + COLS_ADDR_WIDTH
7  ) (
8      // clock input
9      input                                clk,
10     // system reset, low active
11     input                                rstn,
12     // BRAM read interface
13     output [BRAM_ADDR_WIDTH-1:0] bram_rd_addr,
14     input  [7:0] bram_rd_data,
15     // detection control signals
16     input                                detect_start,
17     output                                detect_finish,
18     // detection result related signals
19     output [2:0] detect_peak_num,
20     input  [2:0] disp_peak_idx,
21     output [ROWS_ADDR_WIDTH-1:0] disp_peak_row,
22     output [COLS_ADDR_WIDTH-1:0] disp_peak_col,
23     output [7:0] disp_peak_val
24 );

```

2.2.1) 功能描述

本模块为峰值检测核心模块，负责遍历存储于模块外部 BRAM（Block RAM）中的 64x64 二维数据矩阵。每个数据点均为 8 位无符号整数。模块从中检测出 Top-6 显著峰值。具体规则如下 a)-c)：

a) 峰值数量与排序：

- 最多输出 6 个峰值。若有效峰值不足 6 个，则如实输出实际数量。
- 排序依据为峰值数值大小（从高到低）；如多个峰值数值相同，则地址较低者（BRAM 的地址更小，行优先顺序）优先。

b) 峰值判断标准：

- 幅度阈值：候选点的数值大于 **5×5 邻域的外环**（如下图，绿色环形区域）均值的 2 倍。以下图为例，即满足以下条件：

$$x_{33} > 2 \times \frac{1}{16} \sum_{i=1,5 \text{ or } j=1,5} x_{ij}$$

- 局部极大性：候选点在其 **5×5 邻域**内必须**严格大于其余 24 个邻域点**。

邻域定义：如下图，以当前点为中心（红色），其周围 24 个相邻位置构成 5×5 窗口（蓝色区域+绿色区域）。类似的，3×3 邻域为蓝色区域。

x11	x12	x13	x14	x15
x21	x22	x23	x24	x25
x31	x32	x33	x34	x35
x41	x42	x43	x44	x45
x51	x52	x53	x54	x55

边界处理：对于靠近矩阵边缘、无法构成完整 5×5 邻域的位置，缺失的邻域点不参与计算即可。

	A	B	C	D	E	F	G	H	I	J	K	L
1												
2												
3												
4												
5												
6				x11	x12	x13	x14	x15				
7				x21	x22	x23	x24	x25				
8				x31	x32	x33	x34	x35				
9				x41	x42	x43	x44	x45				
10				x51	x52	x53	x54	x55				
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												
21												
22												
23												
24												
25												
26												
27												
28												

c) 模块输出：

- 检测到的有效峰值总数（≤6）

- 每个峰值对应的坐标（行、列坐标）和数值

2.2.2) BRAM 读取

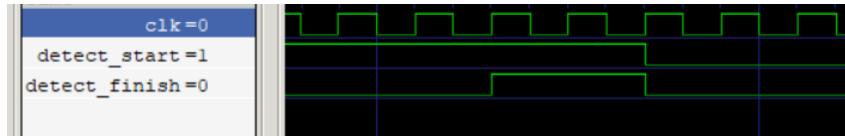
模块需通过 12 位地址 `bram_rd_addr` 来访问模块外部 BRAM 中的数据。此地址信号的高 6 位代表行坐标，低 6 位代表列坐标。

值得注意的是，从 BRAM 中读取数据的过程需要 2 个时钟(`clk`)周期才能完成（详见代码模板中 `modelsim` 子目录下 `tb_bram.vcd` 波形图文件）。在完成读取操作后，将通过 `bram_rd_data` 端口返回一个 8 位无符号整数，即指定地址 `bram_rd_addr` 处存储的数据值。

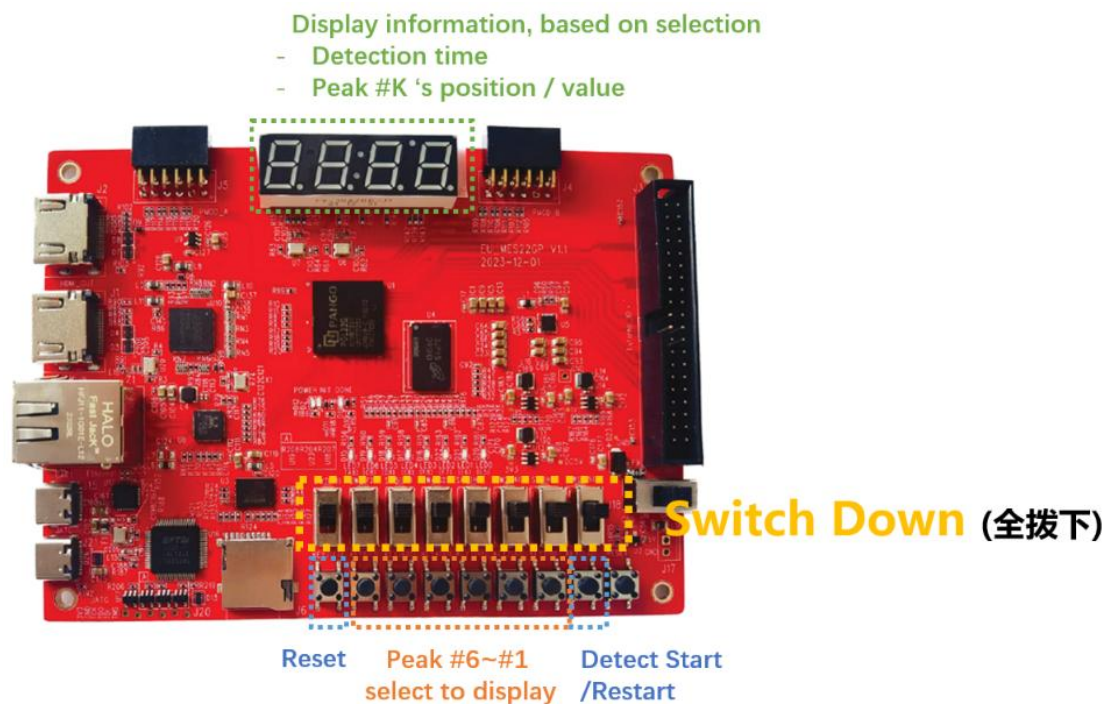
2.2.3) 接口说明

类型	名称	功能说明
input	<code>clk</code>	时钟信号，频率可调
input	<code>rstn</code>	复位信号，低电平有效
output [11:0]	<code>bram_rd_addr</code>	BRAM 读取地址：输出至 BRAM 的读地址，12 位宽（高 6 位为行坐标，低 6 位为列坐标），可寻址 4096 个（64x64）数据点
input [7:0]	<code>bram_rd_data</code>	BRAM 读取数据：从 BRAM 读取的 8 位数据，代表当前地址对应的数值（无符号整数）
input	<code>detect_start</code>	检测启动信号，高电平有效。当 <code>detect_start</code> 从低变高时，模块开始执行峰值检测流程
output	<code>detect_finish</code>	检测完成信号，高电平表示峰值检测已完成，结果已就绪可供查询。该信号在检测结束后拉高，并在下一次 <code>detect_start</code> 有效前或复位时保持低电平
output [2:0]	<code>detect_peak_num</code>	输出检测到的峰值点个数，最多支持检测 6 个峰值（0~6）
input [2:0]	<code>disp_peak_idx</code>	查询的峰值点索引（0~ <code>detect_peak_num</code> -1）。外部逻辑通过该输入来选择查看哪一个峰值的信息
output [5:0]	<code>disp_peak_row</code>	当前峰值（由 <code>disp_peak_idx</code> 指定）所在的行坐标，范围 0-63
output [5:0]	<code>disp_peak_col</code>	当前峰值（由 <code>disp_peak_idx</code> 指定）所在的列坐标，范围 0-63
output [7:0]	<code>disp_peak_val</code>	当前峰值（由 <code>disp_peak_idx</code> 指定）的数值，范围 0-255

注意：`detect_start` 信号在检测过程中，一直维持在高电平。`detect_start` 在检测完成后，会拉回低电平；而与此同时，`detect_finish` 需回到低电平（可以延迟 1 个时钟）。



2.3) FPGA 开发板的交互说明



2.3.1) LED 功能定义

LED 0: 指示检测是否完成 (待实现)

- 熄灭: 待检测
- 闪烁: 检测中
- 常亮: 检测完成

LED1-6: 说明检测峰值数量 (待实现)

- 熄灭: 未完成检测;
- 闪烁: 正在通过按键 (KEY1-KEY6) 查看对应峰值的信息;
- 常亮: 成功识别出该编号的有效峰值 (即存在 Top-X 峰值)

LED 7: 指示系统状态 (给出代码已实现)

- 熄灭: 系统复位中;
- 闪烁: 数据更新中;
- 常亮: 除系统复位和数据更新的其他状态

2.3.2) 轻触开关功能定义

- 按键 0：触发峰值检测的启动或重置操作（即重新开始一次完整的检测流程）。
- 按键 1-6：查看检测到的 Top-1 至 Top-6 的 6 组峰值信息。
 - 默认情况下，按下 按键 1 显示数值最大的峰值（Top-1），按键 2 显示次大峰值（Top-2），依此类推，按峰值大小降序排列。
 - 若未实现完整的峰值排序功能，则允许退化为按 BRAM 地址顺序（即行优先的坐标顺序）输出前 6 个有效峰值：此时，按键 1-6 依次显示按坐标从小到大（先后列）排列的前 6 个满足峰值条件的数据点。
- 按键 7：系统全局复位（对应顶层信号 rstn 输入）

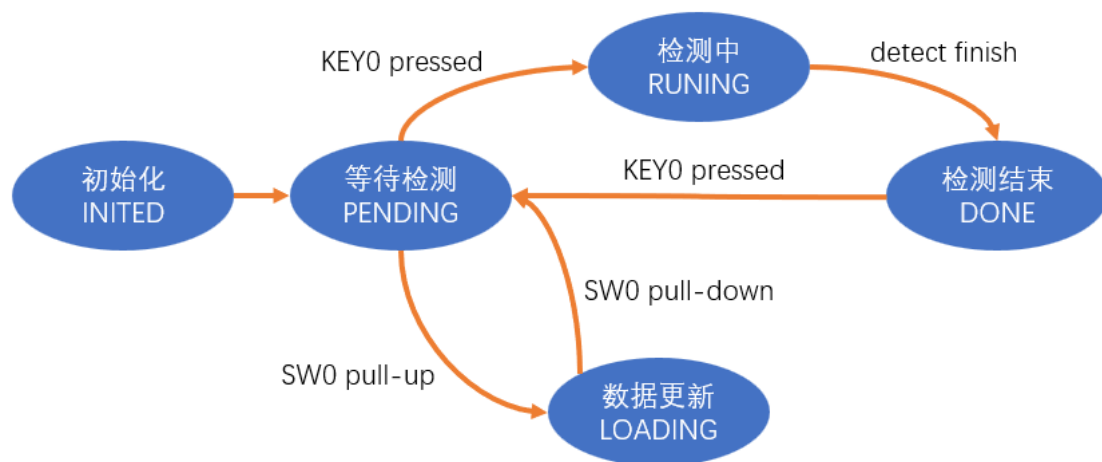
2.3.3) 数码管显示规则

所有数值均以十进制形式在数码管上显示。数码管支持以下 3 种显示模式，即 SegWrapper 模块中输入信号 disp_mode (input [1:0])。

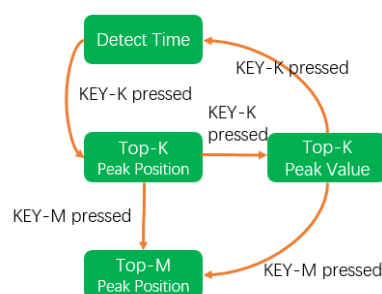
- disp_mode = 0：未完成检测，默认显示全 0
- disp_mode = 1：显示从检测开始到完成所消耗的时间（单位 us）
- disp_mode = 2：显示当前所选峰值坐标（行坐标 X，列坐标 Y），左侧两位数码管显示行坐标 X（0-31），右侧两位数码管显示列坐标 Y（0-31）。
- disp_mode = 3：显示当前所选峰值的数值（8 位无符号整数, 0-255）

2.3.3) 交互流程图

在当前给定的代码中，系统已有以下 4 种状态，即：初始化（INITED）、等待检测（PENDING）、数据更新（LOADING）、检测中（RUNNING）、检测结束（DONE）。其状态转换如下图，通过按键 0（KEY0）、拨动开关（SW7）、Core 模块输出 detect_finish（指示峰值检测已完成）实现状态转换。注意：上述状态机的整体结构和转换逻辑无需修改，请保留原有设计。



当系统进入“检测结束 (DONE)”状态后，数码管应支持在以下三种显示模式之间切换：运行时间（单位： μs ）、峰值坐标（行 X / 列 Y）、峰值数值。该切换逻辑尚未实现，需在现有框架基础上补充相应控制代码。切换方式参考上面按键说明和下图。



三、评分标准

1. FPGA Demo (40 分)

- 交互功能：LED 显示，数码管显示，按键交互（15 分）
- 基础检测：在当前给定数据集（无临近边界的点，峰值点不超过 6 个）进行检测，准确输出结果（不考虑排序）（15 分）
- 扩展检测：评审者更新数据集（有临近边界的点，而且峰值点可能不止 6 个）后，系统依然准确识别峰值（不考虑排序）（5 分）
- 峰值排序：针对基础检测和扩展检测的输出结果，加入峰值排序（5 分）
- **注意：为了检验是否独立完成，会针对代码细节进行现场提问。未能证明是个人独立完成的部分，或无法解释 AI 辅助生成代码的工作原理及逻辑，将不计分数。**

2. 项目报告 (60 分)

- 背景调研（10 分）
- 系统分析与设计（10 分）：阐述整体设计思路，绘制顶层模块框图，详细描述各子模块功能及数据流向，并提供峰值检测算法的流程图。
- 代码实现与逻辑说明（10 分）：针对所有代码修改部分提供详细的设计思路说明，确保评审者能清晰理解设计逻辑与关键算法的硬件化实现方式。
- 子模块仿真验证（10 分）：编写 Testbench 对 Core 模块及 Control 模块进行功能仿真，通过波形分析验证各模块逻辑的正确性。
- UDA 辅助设计（10 分）：记录并说明使用大型语言模型辅助开发/优化代码的过程（如：提示词->LLM 回答->提示调整/优化->...->调试->采纳），以及个人的思考过程。注意：
 - UDA 辅助设计范围限制：大型语言模型（LLM）仅限用于 Core 模块的辅助开发，其余模块（如 Control 模块等）必须由学生独立完成，严禁借助任何大模型工具或平台。
 - 代码理解与去注释要求：需对生成代码进行充分理解与测试，确保掌握其逻辑（Project 答疑仅针对个人编写的代码部分）。此外，生成代码与最终提交代码均不得包含任何注释，以强制要求深入阅读、理解 AI 生成代码，并提升 Demo 现场提问答疑的可信度。

- 其他（10 分）：综合后硬件使用及时钟报告，个人思考与总结。

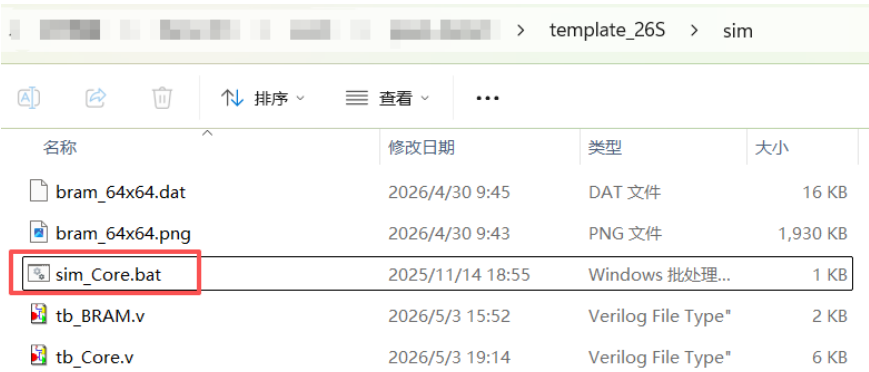
3. Bonus

以下完成一项 Lab 整体得分+1 分，30 分封顶

- 在 15 周 Lab 课堂上 Demo 演示，功能正确（可不含峰值排序），并回答现场提问。
- Demo 演示功能正确（包括峰值排序），而且检测时间（现场记录为准）前 10 名。

Appendix，如何仿真测试 Core 模块

双击 sim 目录下的 sim_Core.bat 脚本（如下图），按照提示继续即可。
详细的仿真过程，需要自己研究 tb_Core.v 代码。



名称	修改日期	类型	大小
bram_64x64.dat	2026/4/30 9:45	DAT 文件	16 KB
bram_64x64.png	2026/4/30 9:43	PNG 文件	1,930 KB
sim_Core.bat	2025/11/14 18:55	Windows 批处理...	1 KB
tb_BRAM.v	2026/5/3 15:52	Verilog File Type"	2 KB
tb_Core.v	2026/5/3 19:14	Verilog File Type"	6 KB

仿真参考结果（如下图）

Right result (WITHOUT boundary check, and sorting) <pre>=== Detection Results === Number of peaks detected: 3 Peak[0]: Row= 8, Col=11, Value=115 Peak[1]: Row=19, Col= 4, Value=128 Peak[2]: Row=47, Col=27, Value=121 === End of Results ===</pre>	Right result (WITHOUT sorting) <pre>=== Detection Results === Number of peaks detected: 5 Peak[0]: Row= 0, Col=18, Value= 99 Peak[1]: Row= 1, Col=34, Value=124 Peak[2]: Row= 8, Col=11, Value=115 Peak[3]: Row=19, Col= 4, Value=128 Peak[4]: Row=47, Col=27, Value=121 === End of Results ===</pre>	Right result (WITH boundary check, and sorting) <pre>=== Detection Results === Number of peaks detected: 5 Peak[0]: Row=19, Col= 4, Value=128 Peak[1]: Row= 1, Col=34, Value=124 Peak[2]: Row=47, Col=27, Value=121 Peak[3]: Row= 8, Col=11, Value=115 Peak[4]: Row= 0, Col=18, Value= 99 === End of Results ===</pre>
--	--	---